

COMPLETE STUDY GUIDE · 2026 EDITION

CLAUDE CERTIFIED ARCHITECT

FOUNDATIONS

Official Exam Preparation · Unofficial Guide



5

Domains

50+

Questions

6

Scenarios

720

Pass Score

Sharvan Kumar

Solution Architect | AI Builder

PREMIUM PDF EDITION

About the Author

Sharvan Kumar is a Solution Architect and AI Builder with hands-on experience designing production systems using the Claude API, Claude Agent SDK, and Model Context Protocol. This guide was built while preparing for the Claude Certified Architect — Foundations exam, combining official exam content with practical exercises and verified terminal output from real coding sessions. All code has been executed and tested.

GitHub: github.com/sharvangkumar | Series: *Building with Claude — A Beginner's Guide*

How to Use This Guide

Covers all five exam domains with theory, code, quizzes, scenario walkthroughs, cheat sheets, and a glossary. Work one domain per session. Use the cheat sheet for final review.

Domain	Topic	Weight
Domain 1	Agentic Architecture & Orchestration	27%
Domain 2	Tool Design & MCP Integration	18%
Domain 3	Claude Code Configuration & Workflows	20%
Domain 4	Prompt Engineering & Structured Output	20%
Domain 5	Context Management & Reliability	15%

Exam format: Multiple choice · 4 of 6 scenarios · Scaled score 100–1000 · Passing score 720 · No penalty for guessing.

DOMAIN 1

Agentic Architecture & Orchestration

27% of scored content

The Agentic Loop

The agentic loop is the core pattern of every Claude agent. Your code sends a message, Claude reasons, and returns either a tool call request or a final answer. Inspect `stop_reason` to drive all control flow.

stop_reason	Meaning	Your code does
"tool_use"	Claude wants to call a tool	Execute tool, append result, send again
"end_turn"	Claude is finished	Read response.content[0].text and exit

Complete Agentic Loop — Production Pattern:

```
while True:
    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=1000, system=system,
        tools=tools, messages=messages
    )
    if response.stop_reason == "end_turn":
        return response.content[0].text # Claude is done
    elif response.stop_reason == "tool_use":
        messages.append({"role":"assistant","content":response.content})
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                result = execute_tool(block.name, block.input)
                tool_results.append({
                    "type":"tool_result",
                    "tool_use_id":block.id,
                    "content":json.dumps(result)
                })
        messages.append({"role":"user","content":tool_results})
```

EXAM TRAP — Loop Termination

WRONG: Checking response text for "task complete" or any natural language signal. WRONG: Using iteration cap as primary stopping mechanism. RIGHT: The **ONLY** correct termination is `stop_reason == "end_turn"`.

Multi-Agent Orchestration

Hub-and-spoke: coordinator manages all inter-agent communication, error handling, and routing. Subagents are specialists with completely isolated context.

Critical rule: Subagents have ZERO automatic context inheritance. Explicitly inject everything they need into their prompt.

PARALLEL: multiple Task calls in ONE coordinator response
coordinator_response = [
 {"type":"tool_use","name":"Task","id":"t1",
 "input":{"subagent":"web_search","prompt":"EU AI regulation 2026"}},
 {"type":"tool_use","name":"Task","id":"t2",
 "input":{"subagent":"doc_analysis","prompt":"Analyse EU AI Act PDF"}}
] # Both execute in parallel -- not sequentially

SDK Hooks

Hook	Fires when	Use case
PostToolUse	After tool executes, before Claude sees result	Normalize timestamps, status codes, currencies
Pre-tool intercept	Before tool executes	Block policy violations — refunds above threshold

KEY PRINCIPLE — Hooks vs Prompt Instructions

Hooks give DETERMINISTIC guarantees. Prompt instructions give PROBABILISTIC compliance. When business rules have financial consequences, always use hooks. Prompt instructions have a non-zero failure rate.

Session Management

Mechanism	Use when
--resume	Prior context is valid — continuing same investigation
fork_session	Exploring divergent approaches from a shared baseline
New session + summary	Prior tool results are stale after code changes

Domain 1 — Practice Questions

Q1. Agent skips get_customer and calls process_refund in 8% of cases. Most effective fix?

A) Add programmatic prerequisite blocking process_refund until get_customer returns a verified ID

B) Enhance system prompt: "always call get_customer before financial operations"

C) Add few-shot examples showing correct tool ordering

D) Implement a classifier restricting tools based on conversation stage

Answer: A — Programmatic enforcement is deterministic. Options B and C are probabilistic with non-zero failure rates.

Q2. Coordinator emits two Task calls in one response. Execution pattern?

A) Second Task waits for first to complete

B) Both subagents execute in parallel simultaneously

C) Coordinator selects most relevant Task

D) Second Task ignored — one Task per turn maximum

Answer: B — Multiple Task calls in one coordinator response = parallel execution.

Q3. Research covers only technology for "global supply chain disruptions." Root cause?

A) Synthesis agent filtered non-technology findings

B) Coordinator task decomposition too narrow — only assigned technology subtasks

C) Web search agent used technology-biased queries

D) Document analysis agent used overly restrictive criteria

Answer: B — Subagents executed their assigned scope correctly. The coordinator assigned the wrong scope.

Q4. What must allowedTools include for coordinator to spawn subagents?

A) Names of all subagent types

B) "Task"

C) "SubAgent"

D) "Delegate"

Answer: B — "Task" is the specific mechanism in Claude Agent SDK for spawning subagents.

Q5. When should you use fork_session over --resume?

A) When prior tool results are stale after code changes

B) When exploring two divergent approaches from a shared baseline simultaneously

C) When the session name is unknown

D) When codebase is too large for one session

Answer: B — fork_session creates independent branches from a shared baseline for divergent exploration.

DOMAIN 2

Tool Design & MCP Integration

18% of scored content

Writing Effective Tool Descriptions

Tool descriptions are the routing table Claude uses for tool selection. Minimal or overlapping descriptions cause misrouting. Include purpose, expected inputs, example queries, and explicit "do not use for" boundaries.

```
# BAD -- causes misrouting
{"name":"analyze_content", "description":"Analyzes content"}
{"name":"analyze_document", "description":"Analyzes documents"}

# GOOD -- clear boundaries
{
  "name": "extract_web_results",
  "description": "Extracts data from web search HTML/JSON.
    USE FOR: raw content from a URL or web page.
    INPUT: Raw HTML, JSON, or plain text from a URL.
    DO NOT USE FOR: PDF files, uploaded docs, database exports."
}
```

Structured Error Responses

Category	Example	Retryable	Agent action
transient	API timeout, service down	Yes	Retry with backoff
validation	Invalid order ID format	No	Reformulate request
business	Refund exceeds limit	No	Communicate policy to customer
permission	Accessing admin record	No	Escalate to authorised agent

MCP Server Configuration

Config File	Scope	Version Controlled	Who sees it
.mcp.json	Project	Yes	Whole team
~/.claude.json	User	No	Personal only

```
# .mcp.json -- env variable keeps secrets safe
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    }
  }
}
```

EXAM TRAP — Tool Count Degradation

Giving an agent 18 tools instead of 4-5 degrades selection reliability. More tools = more decision complexity = more misrouting. Restrict each agent to only the tools needed for its specific role.

Domain 2 — Practice Questions

Q1. Two tools have nearly identical descriptions. Most likely symptom?

- A) Both tools called simultaneously for every request
- B) Model rotates between them unpredictably for similar requests
- C) Error thrown due to ambiguous selection
- D) Model refuses to use either tool

Answer: B — Overlapping descriptions cause unreliable selection. Enrich with clear purpose, input formats, and "do not use for" boundaries.

Q2. MCP tool returns [] for customer with no orders. Use isError: true?

- A) Yes — no results is an error condition
- B) No — empty result is valid success; isError is for access failures
- C) Yes — coordinator needs to know no data returned
- D) Depends on whether customer exists

Answer: B — Empty array = successful query with no matches. Confusing with errors prevents intelligent recovery.

Q3. Developer wants to test experimental MCP server without affecting teammates. Where?

- A) .mcp.json with experimental: true flag
- B) ~/.claude.json — user-scoped, not version controlled
- C) Separate .mcp.experimental.json file
- D) .claude/config.json with personal section

Answer: B — ~/.claude.json is user-scoped and never version controlled.

Q4. tool_choice "any" vs "auto" — key difference?

- A) "any" lets Claude pick any tool; "auto" forces random tool
- B) "any" guarantees a tool call (not text); "auto" may return text instead
- C) "any" uses all tools; "auto" uses only most relevant
- D) They are equivalent

Answer: B — "auto" may return text. "any" guarantees a tool call. For specific tool use {"type":"tool","name":"..."}.

Q5. Agent has 18 tools and 60% selection accuracy. First fix?

- A) Add detailed system prompt instructions about each tool
- B) Add few-shot examples for correct tool selection
- C) Reduce to 4-5 role-relevant tools per agent
- D) Switch to a larger model

Answer: C — Too many tools increases decision complexity. Restrict to role-relevant tools first.

DOMAIN 3

Claude Code Configuration & Workflows

20% of scored content

CLAUDE.md Configuration Hierarchy

Location	Level	Version Controlled	Who sees it
~/.claude/CLAUDE.md	User	No	Only you
CLAUDE.md or .claude/CLAUDE.md	Project	Yes	Whole team
src/api/CLAUDE.md	Directory	Yes	Only in that folder

EXAM TRAP — Most Tested Tran in Domain 3

"A new team member is not getting the coding standards everyone else receives." Root cause: Standards in ~/.claude/CLAUDE.md — user level, not version controlled, not shared. Fix: Move to project-level CLAUDE.md which IS version controlled.

Path-Scoped Rules

```
# .claude/rules/testing.md
# Loads ONLY when editing test files
---
paths: ["**/*.test.*", "**/*.spec.*", "**/tests/**/*.py"]
---
# Testing Conventions
- Use pytest for Python, Jest for TypeScript/React
- Mock ALL external dependencies
- Names: test_[what]_[when]_[expected_result]
```

Skills and Slash Commands

```
# .claude/skills/analyze-codebase/SKILL.md
---
context: fork # isolated sub-agent -- verbose stays in fork
allowed-tools: Read, Grep, Glob # read-only
argument-hint: "Specify scope e.g. authentication flow"
---
Analyze the codebase for: $ARGUMENTS
```

Plan Mode vs Direct Execution

Use Direct Execution	Use Plan Mode
Single file bug fix with clear stack trace	Microservice restructuring across 45+ files
Adding one validation check to one function	Library migration with two valid approaches
Simple, well-scoped, reversible change	Architectural decisions with tradeoffs

Domain 3 — Practice Questions

Q1. Test files scattered throughout codebase. Uniform testing conventions. Best approach?

- A) Add testing conventions to root CLAUDE.md
- B) Create .claude/rules/testing.md with paths: ["**/*.test.*"]
- C) Create CLAUDE.md in every directory with test files
- D) Create a skill that loads testing conventions on demand

Answer: B — Path-scoped rules apply by file type regardless of directory location.

Q2. Skill produces 3,000 tokens of verbose analysis. Keep main conversation clean how?

- A) output: suppress in frontmatter
- B) context: fork in frontmatter
- C) allowed-tools: none in frontmatter
- D) mode: background in frontmatter

Answer: B — context: fork runs skill in isolated sub-agent. Verbose output stays in fork.

Q3. You want /security-audit command for every developer automatically. Where?

- A) ~/.claude/commands/security-audit.md
- B) .claude/commands/security-audit.md
- C) CLAUDE.md with commands: section
- D) .claude/skills/security-audit/SKILL.md

Answer: B — .claude/commands/ is version controlled and automatically available to all developers.

Q4. CI pipeline hangs on: claude "Review this PR". Fix?

- A) Set CLAUDE_HEADLESS=true environment variable
- B) Add the -p (--print) flag for non-interactive mode
- C) Redirect stdin from /dev/null
- D) Add the --batch flag

Answer: B — The -p flag runs Claude Code non-interactively. Processes prompt, writes to stdout, exits.

Q5. You run /memory in Claude Code. What does it do?

- A) Clears all conversation history
- B) Shows which CLAUDE.md files are currently loaded in this session
- C) Saves current session to a named memory slot
- D) Exports conversation history to a file

Answer: B — /memory shows which memory files (CLAUDE.md files) are currently loaded.

DOMAIN 4

Prompt Engineering & Structured Output

20% of scored content

tool_use and JSON Schema

tool_choice	Behaviour	Use when
"auto"	May return text OR call a tool	Tool is optional
"any"	Must call a tool — Claude picks which	Any tool will do
{"type","name"}	Must call THIS specific tool	Guaranteed structured output

```
# Invoice extraction -- guaranteed structure every time
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1000,
    tools=[extraction_tool],
    tool_choice={"type":"tool","name":"extract_invoice_data"},
    messages=[{"role":"user","content":f"Extract: {document}"}]
)
for block in response.content:
    if block.type == "tool_use":
        extracted = block.input # guaranteed JSON object
        # SEMANTIC VALIDATION -- schema cannot catch maths errors
        diff = abs(extracted["total_amount"] - extracted["calculated_total"])
        if diff > 1:
            print(f"Semantic error: {extracted['total_amount']} vs "
                  f"{extracted['calculated_total']}")
```

EXAM TRAP — Syntax vs Semantic Errors

tool_use + JSON schema ELIMINATES: malformed JSON, wrong types, missing required fields. Does NOT PREVENT: line items not summing to total, values in wrong fields, misread amounts. Semantic validation must be in your own code.

Batch Processing API

Property	Value
Cost saving	50% vs synchronous API
Max processing time	Up to 24 hours
Latency SLA	None guaranteed
Multi-turn tool calling	Not supported within a single request
Result correlation	custom_id field

EXAM TRAP — Batch API — Wrong Use Case

NEVER use Batch API for blocking workflows where developers wait for results. No latency SLA = could take up to 24 hours. USE FOR: overnight reports, weekly audits, nightly test generation. NEVER FOR: pre-merge checks, real-time user-facing responses.

Domain 4 — Practice Questions

Q1. You set tool_choice "auto". Sometimes Claude returns text instead of JSON. Why?

- A) Tool description is too long
- B) "auto" allows Claude to decide — may return text; use "any" or forced selection
- C) Document exceeded context window
- D) JSON schema has an error

Answer: B — "auto" is not a guarantee. For guaranteed structured output use {"type": "tool", "name": "..."}.

Q2. Schema has vendor_address as required. 30% of invoices have no address. What happens?

- A) Extraction fails with schema validation error
- B) Claude returns null for the field
- C) Claude fabricates an address to satisfy the required constraint
- D) Claude asks user for the missing address

Answer: C — Required fields with no information in source cause fabrication. Use nullable: type: ["string", "null"].

Q3. tool_use + JSON schema guarantees what; does NOT prevent what?

- A) Guarantees correct values; does not guarantee field names
- B) Guarantees syntax (valid JSON, correct types); does not prevent semantic errors
- C) Guarantees both syntax and semantics
- D) Guarantees no hallucination; does not prevent format errors

Answer: B — Schema enforces structure. It cannot detect that line items do not sum to the stated total.

Q4. 500 documents submitted in batch. 13 fail. How to resubmit?

- A) Resubmit all 500 with modified prompt
- B) Identify 13 by custom_id and resubmit only those with modifications
- C) Increase max_tokens and resubmit all 500
- D) Mark 13 as exceptions and proceed

Answer: B — custom_id identifies each request-response pair. Resubmit only failed documents.

Q5. When to use few-shot examples instead of detailed instructions?

- A) Always — examples are always more effective
- B) When instructions alone produce inconsistent results and you need to demonstrate judgment
- C) Only when task involves code rather than language
- D) Only when model is unfamiliar with the domain

Answer: B — Few-shot examples most effective when instructions produce inconsistent output.

DOMAIN 5

Context Management & Reliability

15% of scored content

The Lost-in-the-Middle Effect

Claude has no persistent memory between API calls. Every call receives the full conversation history. LLMs have highest attention at beginning and end of input — information buried in the middle gets lower recall accuracy.

Day 1 Exercise — Verified Terminal Output:

```
Full API response : 45 fields, 1758 characters
Trimmed response : 5 fields, 144 characters
Reduction : 92%

One conversation, 5 tool calls:
Full load --> 8,790 characters of noise
Trimmed --> 720 characters of signal
```

The Case Facts Block Solution

```
case_facts = """
<case_facts>
ORDER_ID: ORD-8821
REFUND_AMOUNT: Rs.4750
EMAIL_ORIGINAL: sharma.old@gmail.com
EMAIL_CORRECTED: sharvan.kumar@infosys.com
</case_facts>
"""

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    system=case_facts, # position 1 -- always -- never buried
    messages=messages
)
```

Verbose Tool Output Trimming

```
def trim_for_damage_return(full_response: dict) -> dict:
    # 45 fields -> 5 fields | 1758 chars -> 144 chars | 92% reduction
    return {
        "order_id": full_response["order_id"],
        "status": full_response["status_label"],
        "total_amount": full_response["total_amount"],
        "return_eligible": full_response["return_eligible"],
        "return_window_closes": full_response["return_window_closes"]
    }
# ALWAYS trim BEFORE appending to conversation history
```

Error Propagation Anti-Patterns

Anti-Pattern	Problem
Return [] as success on timeout	Coordinator cannot distinguish failure from valid empty result
Generic "search unavailable"	Coordinator cannot decide: retry? alternative? proceed?
Terminate on single failure	Discards all successful subagent work unnecessarily

Domain 5 — Practice Questions

Q1. Agent misremembers refund amounts after 15+ turn conversations. Most effective fix?

- A) Switch to model with larger context window
- B) Extract critical facts into persistent block in system prompt, outside summarized history
- C) Summarize conversation every 5 turns
- D) Repeat facts at end of every user message

Answer: B — System prompt is always position 1 where attention is highest. Never gets buried.

Q2. Subagent returns {"results":[]} with isError: false on timeout. Problem?

- A) Empty arrays should use null instead
- B) Coordinator cannot distinguish timeout from valid search with no results — silent failure
- C) Subagent should have retried before returning
- D) isError should always be included

Answer: B — Returning empty as success on timeout is silent suppression anti-pattern.

Q3. 97% overall extraction accuracy. Manager proposes removing human review. What first?

- A) Verify sample size was statistically significant
- B) Verify accuracy is consistent across all document types and fields
- C) Verify the 3% errors are evenly distributed
- D) Verify latest model version was used

Answer: B — 97% overall can mask 60% failure on specific document types. Segment accuracy first.

Q4. After 30 turns Claude references "typical patterns" instead of specific classes. Cause?

- A) Claude reached knowledge cutoff for this codebase
- B) Context degradation — earlier findings in low-attention zone; use scratchpad files
- C) Model is hallucinating; add "do not make up information"
- D) Session corrupted: use --resume to reload

Answer: B — Context degradation causes fallback to generic descriptions. Fix: scratchpad files, /compact, or subagents with injected summaries.

Q5. Customer says "I want to speak to a human agent right now." What should agent do?

- A) Acknowledge, then attempt to resolve since issue is straightforward
- B) Escalate immediately without attempting investigation
- C) Ask why they want a human before deciding
- D) Offer resolution first, escalate after three failed attempts

Answer: B — Explicit customer request = non-negotiable escalation trigger. Escalate immediately.

All 6 Exam Scenarios

The exam presents 4 scenarios randomly selected from these 6. Each frames a set of multiple-choice questions.

Scenario 1: Customer Support Resolution Agent

Primary domains: Domains 1, 2, 5

Support agent with tools: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`.

- Programmatic prerequisites blocking `process_refund` until `get_customer` succeeds
- `PostToolUse` hooks normalizing heterogeneous tool output formats
- Structured escalation summaries for human agents lacking conversation access
- Explicit escalation triggers: customer request, policy gaps, inability to progress

Scenario 2: Code Generation with Claude Code

Primary domains: Domains 3, 5

Claude Code for code generation, refactoring, debugging, documentation.

- Project vs user-level `CLAUDE.md` and version control sharing
- Path-scoped rules for conventions spanning multiple directories
- Plan mode for complex multi-file architectural tasks
- context: fork for verbose skill output isolation

Scenario 3: Multi-Agent Research System

Primary domains: Domains 1, 2, 5

Coordinator delegates to: web search, document analysis, synthesis, report generation agents.

- Coordinator task decomposition breadth — narrow = missing topic areas
- Explicit context injection: subagents have zero automatic context inheritance
- Structured claim-source mappings preserved through synthesis steps
- Parallel subagent execution via multiple `Task` calls in one response

Scenario 4: Developer Productivity with Claude

Primary domains: Domains 2, 3, 1

Agent helps engineers explore codebases, understand legacy systems, generate boilerplate.

- `Grep` (content search) vs `Glob` (file path pattern matching)
- Incremental codebase exploration starting with entry points
- context: fork for isolating verbose discovery output
- Scratchpad files for persisting key findings across context boundaries

Scenario 5: Claude Code for Continuous Integration

Primary domains: Domains 3, 4

Integrating Claude Code into CI/CD pipelines for automated code reviews.

- `-p` flag for non-interactive CI mode
- `--output-format json` + `--json-schema` for programmatic PR comment posting
- Include prior review findings to avoid duplicate comments on re-runs
- `CLAUDE.md` for CI context: testing standards, fixture conventions

Scenario 6: Structured Data Extraction

Primary domains: Domains 4, 5

Extracting from unstructured documents, validating with JSON schemas.

- tool_use with JSON schema for guaranteed structure
- Nullable fields preventing fabrication of absent information
- Semantic validation: calculated_total vs stated_total
- Accuracy segmentation by document type before removing human review

Quick Reference Cheat Sheet

stop_reason Values

tool_use Claude wants a tool — execute and loop
end_turn Claude is done — read response and exit

tool_choice Options

auto May return text — NOT guaranteed tool call
any Must call a tool — Claude picks which
{type,name} Must call THIS specific tool — guaranteed

CLAUDE.md Levels

~/.claude/ User — not version controlled — you only
root CLAUDE.md Project — version controlled — whole team
subdir CLAUDE.md Directory — loads only in that folder

MCP Config

.mcp.json Project — version controlled — shared team
~/.claude.json User — not controlled — personal only
\${TOKEN} Env variable expansion — never commit secrets

Error Categories

transient Timeout — isRetryable: true
validation Bad input — isRetryable: false
business Policy violation — isRetryable: false
permission Access denied — isRetryable: false

Batch API

Cost 50% savings vs synchronous
Window Up to 24 hours processing
SLA None — not for blocking workflows
Tools No multi-turn tool calling in request

Claude Code CLI

-p Non-interactive mode for CI/CD
--output-format json Machine-parseable output
--json-schema Enforce output structure
--resume Continue named previous session

Built-in Tools

Grep Search file CONTENTS for patterns
Glob Find files by NAME/extension pattern
Read/Write Full file operations
Edit Targeted mods — unique text match

Top 10 Exam Traps

#	Trap	Wrong Answer	Correct Answer
1	Financial rule enforcement	Add system prompt instruction	Programmatic hook / prerequisite gate
2	Subagent context	Inherits coordinator context automatically	Must explicitly inject context in prompt
3	Batch API for blocking workflows	50% savings justifies it	No latency SLA — async only
4	tool_choice "auto"	Guarantees a tool call	May return text — use "any" or forced selection
5	Research coverage gaps	Blame synthesis / search agent	Coordinator task decomposition too narrow
6	97% aggregate accuracy	Safe to remove human review	Segment by doc type and field first
7	Larger context window	Fixes attention quality	Multi-pass review fixes depth — not window
8	User-level CLAUDE.md	Shared via version control	Never version controlled — you only
9	JSON schema + tool_use	Prevents all errors	Eliminates syntax only — semantic errors remain
10	Loop termination	Check text or use iteration cap	Only stop_reason=="end_turn" is correct

Glossary of Key Terms

stop_reason

Signal from Claude why generation stopped. "tool_use" = wants to call a tool. "end_turn" = finished.

tool_use

When Claude decides to call a tool, it returns a block of type "tool_use" with tool name and parameters.

tool_result

The message you send back to Claude after executing a tool. Contains the tool output.

tool_choice

API parameter: "auto" (may return text), "any" (must call a tool), {type,name} (must call specific tool).

Agentic loop

Cycle: Claude reasons, requests tool, you execute, you return result, Claude reasons again. Repeats until end_turn.

MCP

Model Context Protocol — standard interface for connecting Claude to external tools and data sources.

Coordinator

Claude instance managing workflow — task decomposition, delegation, aggregation, error routing.

Subagent

Separate Claude instance for a specific subtask. Isolated context — zero automatic inheritance.

Task tool

Mechanism in Claude Agent SDK for coordinator to spawn subagents. Must be in allowedTools.

PostToolUse hook

Intercepts tool results after execution, before Claude processes them. Used for data normalization.

context: fork

SKILL.md frontmatter running skill in isolated sub-agent. Verbose output stays in the fork.

CLAUDE.md

Configuration file Claude Code reads automatically. Contains project conventions and standards.

Path-scoped rules

.claude/rules/ files with YAML frontmatter paths. Load only when editing matching files.

@import

CLAUDE.md syntax for referencing external files, keeping configuration modular.

Plan mode

Claude Code mode for exploring and proposing without making changes. For complex tasks.

fork_session

Creates independent branches from shared baseline to explore divergent approaches.

Lost in the middle

LLMs have lower attention fidelity for information in the middle of long inputs.

Case facts block

Critical facts pinned in system prompt at position 1 — never buried in history.

isError flag

MCP tool response flag signaling failure to the agent.

isRetryable

Boolean in structured error responses — whether retrying the same request is appropriate.

custom_id

Field in Message Batches API correlating each request with its response.

JSON Schema

Standard for describing JSON structure. Used in tool input_schema.

Semantic error

Logically incorrect data that passes schema validation — e.g., line items not summing to total.

Nullable field

Schema field with type: ["string", "null"] — Claude returns null when information is absent.

Few-shot prompting

2–4 concrete examples demonstrating desired output format and judgment.

Hub-and-spoke

Multi-agent architecture where coordinator manages all inter-agent communication.

/compact

Claude Code command to reduce context usage during extended sessions.

allowedTools

Restricts which tools agent can invoke. Must include "Task" to spawn subagents.

Online References

Resource	URL	Covers
Claude API — Messages	docs.anthropic.com/en/api/messages	tool_use, stop_reason, tool_choice
Tool Use Guide	docs.anthropic.com/en/docs/build-with-claude/tool-use	JSON schema, reading tool_use responses
Prompt Engineering	docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview	Few-shot, structured output
Claude Code	docs.anthropic.com/en/docs/claude-code/overview	CLAUDE.md, skills, plan mode, CI/CD
Model Context Protocol	modelcontextprotocol.io	MCP spec, server config, tool design
Message Batches API	docs.anthropic.com/en/docs/build-with-claude/message-batches	Batch processing, custom_id
Claude Agent SDK	docs.anthropic.com/en/docs/agents-and-tools/claude-agent-sdk	AgentDefinition, Task tool, hooks
Anthropic Console	console.anthropic.com	API keys, usage monitoring, billing
GitHub — Day 1	github.com/sharvankumar/claude-architect-prep/tree/main/day1	Context management exercises
GitHub — Day 2	github.com/sharvankumar/claude-architect-prep/tree/main/day2	Structured output exercises